then $\tilde{\mathbf{P}}$, $\tilde{\mathbf{Q}}$, $\tilde{\mathbf{R}}$, $\tilde{\mathbf{S}}$, which have the same sizes as $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{S}$, respectively, can be found by either the formulas

$$\begin{aligned}
\tilde{\mathbf{P}} &= (\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1} \\
\tilde{\mathbf{Q}} &= -(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}\cdot(\mathbf{Q}\cdot\mathbf{S}^{-1}) \\
\tilde{\mathbf{R}} &= -(\mathbf{S}^{-1}\cdot\mathbf{R})\cdot(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1} \\
\tilde{\mathbf{S}} &= \mathbf{S}^{-1} + (\mathbf{S}^{-1}\cdot\mathbf{R})\cdot(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}\cdot(\mathbf{Q}\cdot\mathbf{S}^{-1})
\end{aligned} \tag{2.7.24}$$

or else by the equivalent formulas

$$\begin{aligned}
\tilde{\mathbf{P}} &= \mathbf{P}^{-1} + (\mathbf{P}^{-1}\cdot\mathbf{Q})\cdot(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}\cdot(\mathbf{R}\cdot\mathbf{P}^{-1}) \\
\tilde{\mathbf{Q}} &= -(\mathbf{P}^{-1}\cdot\mathbf{Q})\cdot(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1} \\
\tilde{\mathbf{R}} &= -(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}\cdot(\mathbf{R}\cdot\mathbf{P}^{-1}) \\
\tilde{\mathbf{S}} &= (\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}
\end{aligned} \tag{2.7.25}$$

The parentheses in equations (2.7.24) and (2.7.25) highlight repeated factors that you may wish to compute only once. (Of course, by associativity, you can instead do the matrix multiplications in any order you like.) The choice between using equations (2.7.24) and (2.7.25) depends on whether you want $\tilde{\mathbf{P}}$ or $\tilde{\mathbf{S}}$ to have the simpler formula; or on whether the repeated expression $(\mathbf{S}-\mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q})^{-1}$ is easier to calculate than the expression $(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R})^{-1}$; or on the relative sizes of $\mathbf{P}$ and $\mathbf{S}$; or on whether $\mathbf{P}^{-1}$ or $\mathbf{S}^{-1}$ is already known.

Another sometimes useful formula is for the determinant of the partitioned matrix,

$$\det\mathbf{A} = \det\mathbf{P}\,\det(\mathbf{S} - \mathbf{R}\cdot\mathbf{P}^{-1}\cdot\mathbf{Q}) = \det\mathbf{S}\,\det(\mathbf{P} - \mathbf{Q}\cdot\mathbf{S}^{-1}\cdot\mathbf{R}) \tag{2.7.26}$$

2.7.5 Indexed Storage of Sparse Matrices

We have already seen (§2.4) that tri- or band-diagonal matrices can be stored in a compact format that allocates storage only to elements that can be nonzero, plus perhaps a few wasted locations to make the bookkeeping easier. What about more general sparse matrices? When a sparse matrix of dimension $M \times N$ contains only a few times M or N nonzero elements (a typical case), it is surely inefficient — and often physically impossible — to allocate storage for all MN elements. Even if one did allocate such storage, it would be inefficient or prohibitive in machine time to loop over all of it in search of nonzero elements.

Obviously some kind of indexed storage scheme is required, one that stores only nonzero matrix elements, along with sufficient auxiliary information to determine where an element logically belongs and how the various elements can be looped over in common matrix operations. Unfortunately, there is no one standard scheme in general use. Each scheme has its own pluses and minuses, depending on the application.

Before we look at sparse matrices, let's consider the simpler problem of a *sparse vector*. The obvious data structure is a list of the nonzero values and another list of the corresponding locations:

sparse.h

```
struct NRsparseCol
Sparse vector data structure.
{
    Int nrows;                  Number of rows.
    Int nvals;                  Maximum number of nonzeros.
```

```
    VecInt row_ind;                                  Row indices of nonzeros.
    VecDoub val;                                     Array of nonzero values.

    NRsparseCol(Int m,Int nnvals) : nrows(m), nvals(nnvals),
    row_ind(nnvals,0),val(nnvals,0.0) {}             Constructor. Initializes vector to zero.

    NRsparseCol() : nrows(0),nvals(0),row_ind(),val() {}     Default constructor.

    void resize(Int m, Int nnvals) {
        nrows = m;
        nvals = nnvals;
        row_ind.assign(nnvals,0);
        val.assign(nnvals,0.0);
    }

};
```

While we think of this as defining a column vector, you can use exactly the same data structure for a row vector — just mentally interchange the meaning of row and column for the variables. For matrices, however, we have to decide ahead of time whether to use row-oriented or column-oriented storage.

One simple scheme is to use a vector of sparse columns:

```
NRvector<NRsparseCol *> a;
for (i=0;i<n;i++) {
    nvals=...
    a[i]=new NRsparseCol(m,nvals);
}
```

Each column is filled with statements like

```
count=0;
for (j=...) {
    a[i]->row_ind[count]=...
    a[i]->val[count]=...
    count++;
}
```

This data structure is good for an algorithm that primarily works with columns of the matrix, but it is not very efficient when one needs to loop over all elements of the matrix.

A good general storage scheme is the *compressed column storage* format. It is sometimes called the Harwell-Boeing format, after the two large organizations that first systematically provided a standard collection of sparse matrices for research purposes. In this scheme, three vectors are used: `val` for the nonzero values as they are traversed column by column, `row_ind` for the corresponding row indices of each value, and `col_ptr` for the locations in the other two arrays that start a column. In other words, if `val[k]=a[i][j]`, then `row_ind[k]=i`. The first nonzero in column `j` is at `col_ptr[j]`. The last is at `col_ptr[j+1]-1`. Note that `col_ptr[0]` is always 0, and by convention we define `col_ptr[n]` equal to the number of nonzeros. Note also that the dimension of the `col_ptr` array is $N + 1$, not N. The advantage of this scheme is that it requires storage of only about two times the number of nonzero matrix elements. (Other methods can require as much as three or five times.)

As an example, consider the matrix

$$\begin{bmatrix} 3.0 & 0.0 & 1.0 & 2.0 & 0.0 \\ 0.0 & 4.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 7.0 & 5.0 & 9.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 6.0 & 5.0 \end{bmatrix} \tag{2.7.27}$$

In compressed column storage mode, matrix (2.7.27) is represented by two arrays of length 9 and an array of length 6, as follows

index k	0	1	2	3	4	5	6	7	8
val[k]	3.0	4.0	7.0	1.0	5.0	2.0	9.0	6.0	5.0
row_ind[k]	0	1	2	0	2	0	2	4	4

(2.7.28)

index i	0	1	2	3	4	5
col_ptr[i]	0	1	3	5	8	9

Notice that, according to the storage rules, the value of N (namely 5) is the maximum valid index in `col_ptr`. The value of `col_ptr[5]` is 9, the length of the other two arrays. The elements 1.0 and 5.0 in column number 2, for example, are located in positions `col_ptr[2]` $\le k <$ `col_ptr[3]`.

Here is a data structure to handle this storage scheme:

sparse.h
```
struct NRsparseMat
Sparse matrix data structure for compressed column storage.
{
    Int nrows;                          Number of rows.
    Int ncols;                          Number of columns.
    Int nvals;                          Maximum number of nonzeros.
    VecInt col_ptr;                     Pointers to start of columns. Length is ncols+1.
    VecInt row_ind;                     Row indices of nonzeros.
    VecDoub val;                        Array of nonzero values.

    NRsparseMat();                          Default constructor.
    NRsparseMat(Int m,Int n,Int nnvals);    Constructor. Initializes vector to zero.
    VecDoub ax(const VecDoub &x) const;     Multiply A by a vector x[0..ncols-1].
    VecDoub atx(const VecDoub &x) const;    Multiply A^T by a vector x[0..nrows-1].
    NRsparseMat transpose() const;          Form A^T.
};
```

The code for the constructors is standard:

sparse.h
```
NRsparseMat::NRsparseMat() : nrows(0),ncols(0),nvals(0),col_ptr(),
    row_ind(),val() {}
NRsparseMat::NRsparseMat(Int m,Int n,Int nnvals) : nrows(m),ncols(n),
    nvals(nnvals),col_ptr(n+1,0),row_ind(nnvals,0),val(nnvals,0.0) {}
```

The single most important use of a matrix in compressed column storage mode is to multiply a vector to its right. Don't implement this by traversing the rows of $\mathbf{A}$, which is extremely inefficient in this storage mode. Here's the right way to do it:

sparse.h
```
VecDoub NRsparseMat::ax(const VecDoub &x) const {
    VecDoub y(nrows,0.0);
    for (Int j=0;j<ncols;j++) {
        for (Int i=col_ptr[j];i<col_ptr[j+1];i++)
            y[row_ind[i]] += val[i]*x[j];
    }
    return y;
}
```

Some inefficiency occurs because of the indirect addressing. While there are other storage modes that minimize this, they have their own drawbacks.

It is also simple to multiply the *transpose* of a matrix by a vector to its right, since we just traverse the columns directly. (Indirect addressing is still required.) Note that the transpose matrix is not actually constructed.